Deep Learning in Computer Vision· Lecture 1 of 9

Learning & Neural Network Foundations

From machine learning to the fully connected network

Agenda

- What does it mean for a computer to **learn**?
- The four **types of learning**
- The artificial **neuron** and activation functions
- **Fully connected** (feed-forward) networks
- Why fully connected networks struggle with images
- How a network is **trained** (preview of Lecture 2)

Learning

- Learning is how a machine uses **data** to improve at a task **without being explicitly programmed** for it.
- In computer vision the data are **images and videos**.
- Like a baby: learns from experience and observation, not from a rulebook.

Why Computers Need Learning

- Computers know nothing about the world by default.
- For tasks like **recognizing faces** or **diagnosing disease**, writing every rule by hand is impossible.
- Learning lets computers gain knowledge from **data** and human experience.

Four Types of Learning

1) Supervised

Learns from **labeled** examples, like a teacher correcting answers.

CV: tumor yes/no.

2) Unsupervised

No labels; finds structure by itself (grouping similar toys).

CV: clustering images.

3) Semi-supervised

Few labels + many unlabeled images.

CV: medical scans labeled by few doctors.

4) Reinforcement

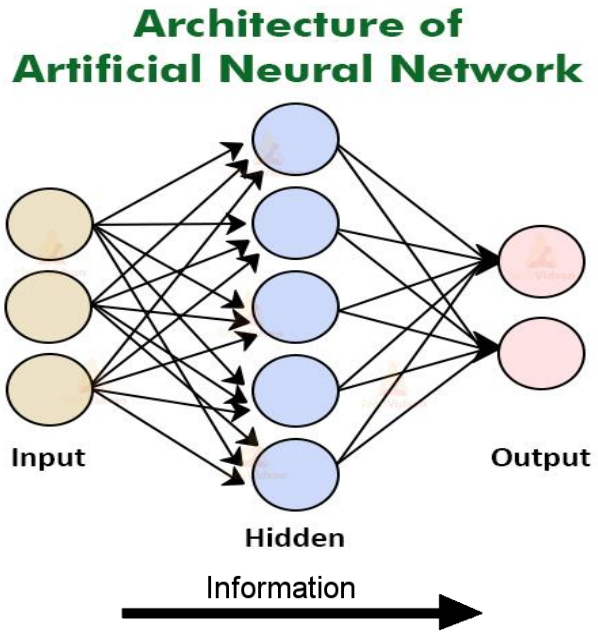
Trial and error with rewards/penalties.

Example: robot learning to walk.

Artificial Neural Networks

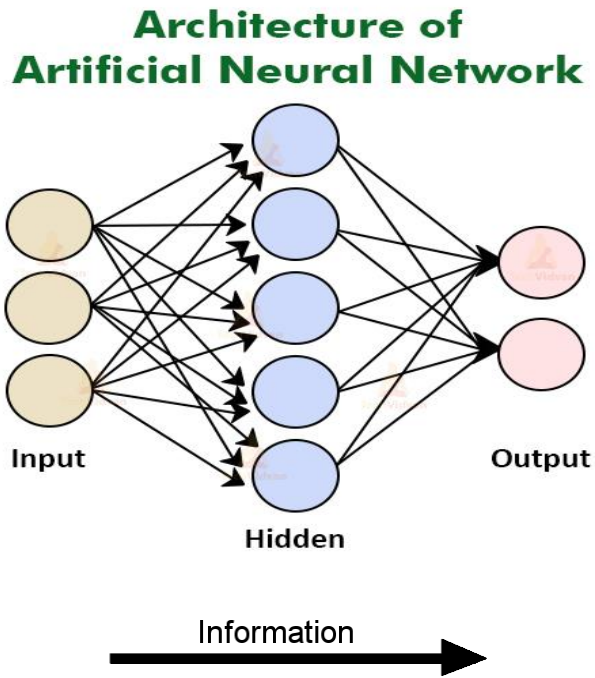
- ANNs copy two ideas from biological nets:
- **Neurons** (nodes): simple processing units
- **Synapses** (weights): connection strengths that are **learned**
- Simple principles, **very complex behaviour**
- Used as problem solvers
    - e.g. COVID-19 vs. normal from X-ray

- Information flow is unidirectional (e.g. Feed-Forward Nets)
    - Data is presented to *Input layer*
    - Passed on to *Hidden Layer*
    - Passed on to *Output layer*
- Information is distributed
- Information processing is parallel



Feed-forward (Fully Connected) Networks

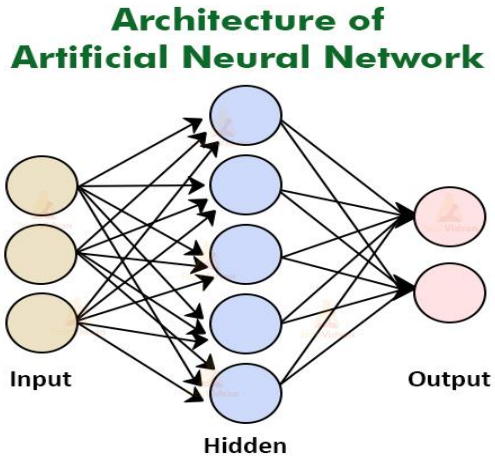
- Information flows **one way**: input → hidden → output
- Input layer holds data, e.g. **pixel intensities**
- Every neuron connects to **every** neuron in the next layer
- One layer in matrix form: $\boldsymbol{h} = \boldsymbol{f}(\boldsymbol{W}\,\boldsymbol{x} + \boldsymbol{b})$
- Abstract data needs encoding (e.g. class "Chris" = 0-0-1)



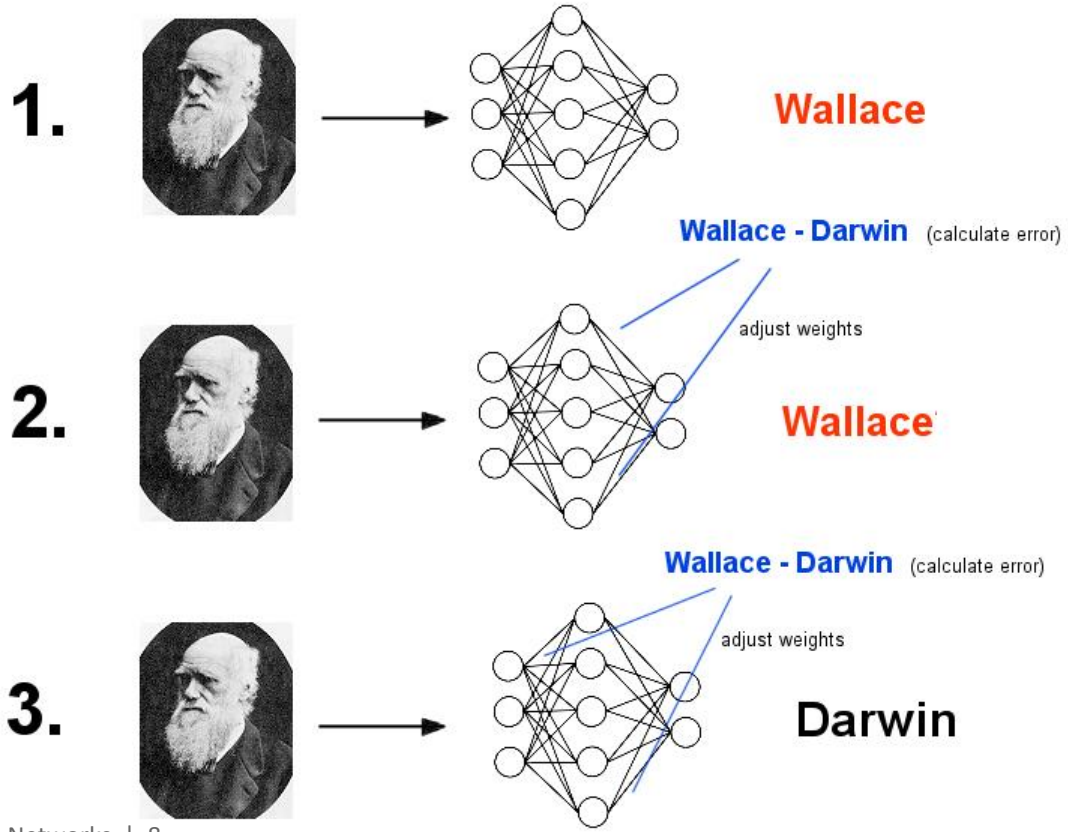
Backpropagation

- Requires training set (input / output pairs)
- Starts with small random weights
- Error is used to adjust weights (supervised learning)

→ Repeat the process



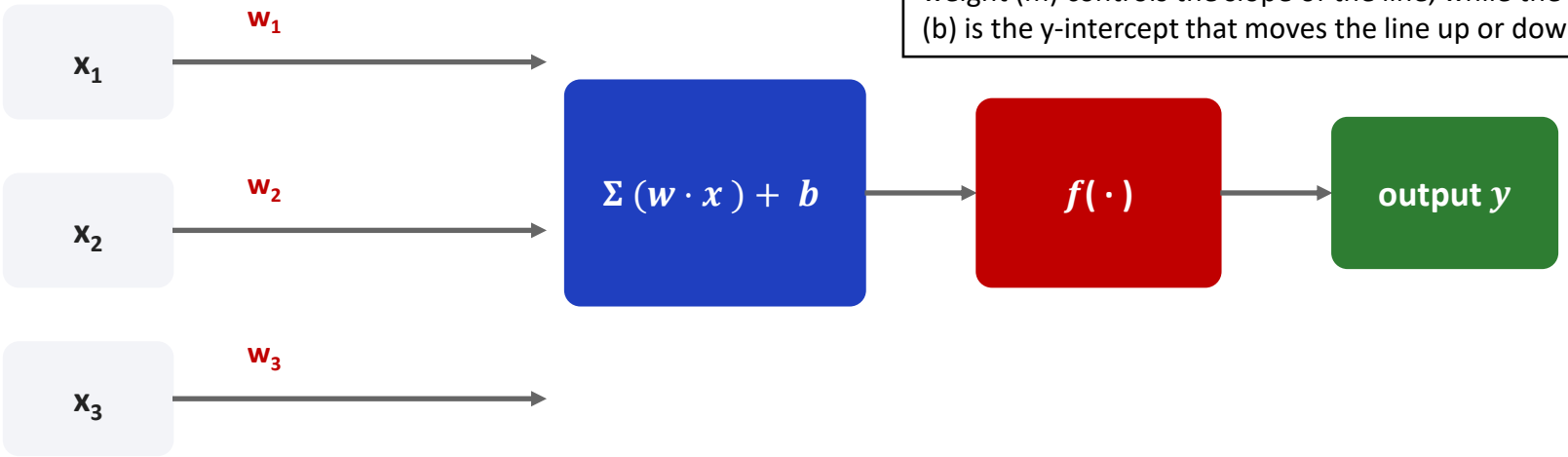
Example: Training the Network – Learning (Charles Darwin and Alfred Russel Wallace, two notable figures in the history of evolutionary theory)



The Artificial Neuron x_i

We need a bias because it gives the network the **flexibility to shift its decision boundary** up, down, left, or right

Think of the classic linear equation y = mx + b. The weight (m) controls the slope of the line, while the bias (b) is the y-intercept that moves the line up or down.



$$y = f(w_1 \cdot x_1 + w_2 \cdot x_2 + w_3 \cdot x_3 + b)$$

- Weights w and bias b are the parameters the network learns;
- f is the activation function.

Activation Non-Linear Functions $f(.)$

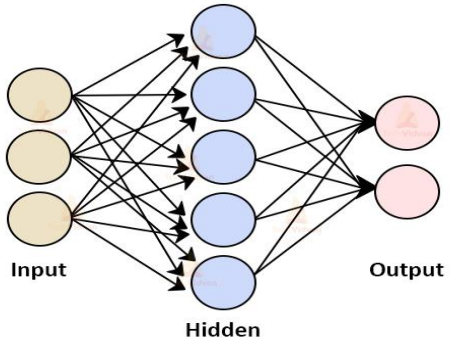
Name	Formula	Range	Remark
Sigmoid	$f(x) = \frac{1}{1+e^{-x}}$	(0, 1)	• Maps any real-valued input *x* into a probability range between 0 and 1. • saturates, gradients vanish
Hyperbolic Tangent (Tanh)	$f(x) = \frac{e^{x}-e^{-x}}{e^{x}+e^{-x}}$	(-1, 1)	• Rescaled sigmoid function centered at zero • Zero-centred; still saturates
Rectified Linear Unit (ReLU)	$f(x) = \max(0, x)$	[0, ∞)	• Returns *x* if *x* > 0, and returns 0 otherwise • Default in CNNs; cheap, no saturation for x > 0
Softmax	$\sigma(\boldsymbol{x})_i = \frac{e^{x_i}}{\sum_{j=1}^{K} e^{x_j}}$ $\mathbf{x} = [x_1, x_2, \ldots, x_K]^T$	(0, 1), sums to 1	• Used primarily in the output layer of multi-class classification neural networks to convert raw unnormalized scores (logits) into a probability distribution that sums to 1

Counting Parameters

Layer with n inputs and m neurons has $n \times m$ weights + m biases

Network	Calculation	Parameters
4 → 3 → 2 (toy network)	4×3+3 (Hidden Layer) + 3×2+2 (Output Layer)	23
28×28 digit → 128 → 10	784×128+128 (Hidden) + 128×10+10 (Output)	101,770
224×224×3 image → 1000	150,528×1000 + 1000	150.5 million

One layer on a modest photo already has 150 million weights!!!



Why Fully Connected Nets Fail on Images

Too many weights

A 224×224 RGB image is **150,528 inputs**. Huge memory, easy overfitting.

No spatial structure

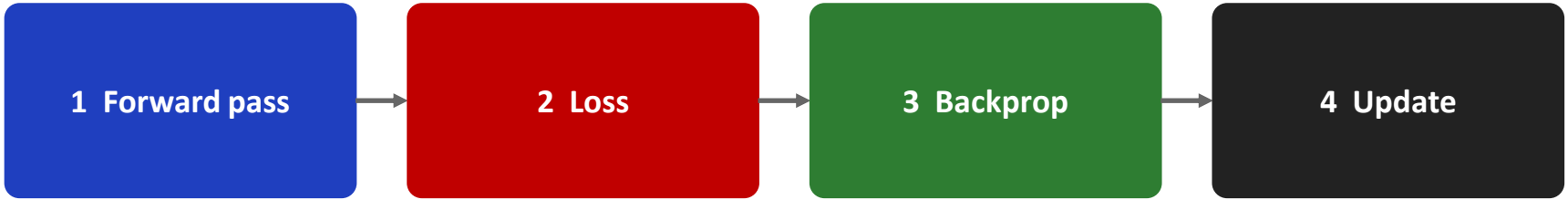
Flattening pixels **throws away neighbourhoods**. Nearby pixels are not treated as related. In other words, Vectorizing 2D pixel grids into 1D vectors treats adjacent pixels identically to pixels separated across opposite corners of the image.

Not shift-invariant

Dense weights are tied to exact pixel positions. If an object shifts by 5 pixels, a completely separate set of weights must learn to detect it. For example, a cat in an image moved a few pixels looks like **different input**; it must be learned again.

Upcoming lecture 4: convolutional layers (CNN) fix all three.

Training a Network (Big Picture)



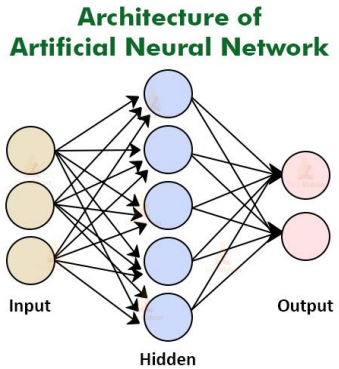
Input → prediction

Compare with label

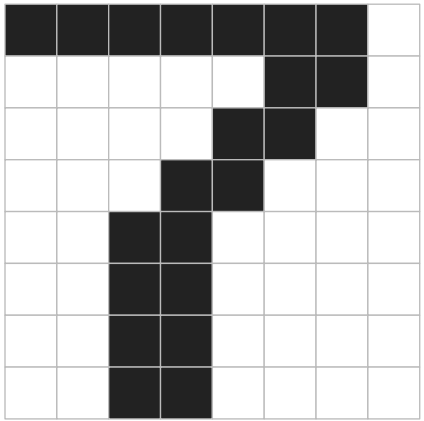
Gradient of loss for each weight

Adjust weights; repeat

Start with small random weights.
Repeat over the training set many times.



Example: Recognizing Handwritten Digits



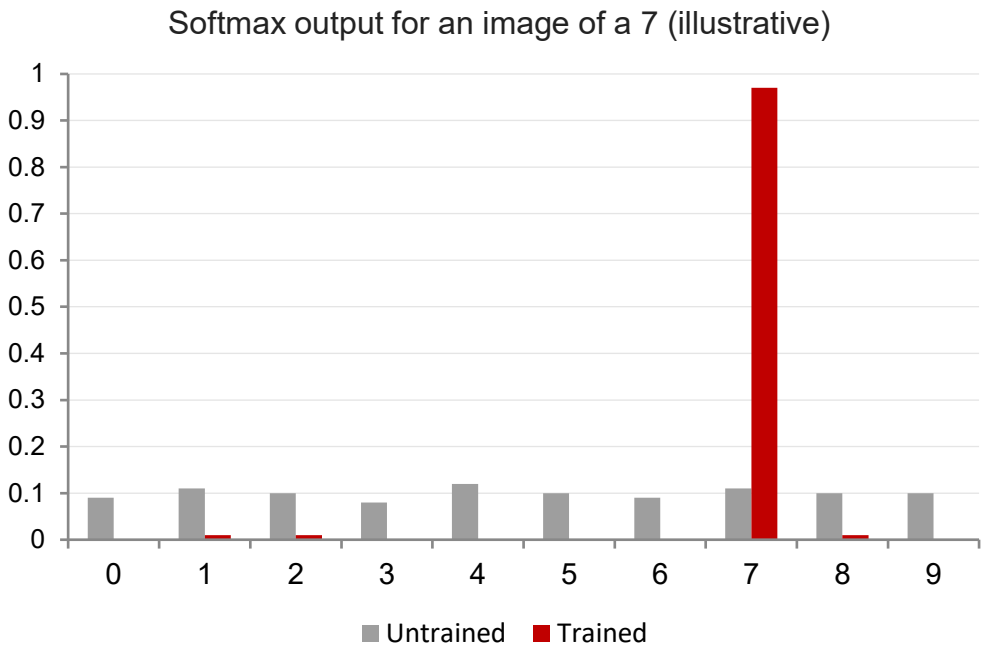
28×28 image (8×8 shown)



Parameters: 784×128+128 + 128×10+10 = **101,770**

Output k = probability that the digit is k. Training uses cross-entropy loss, backprop and an optimizer (**Next lecture**).

Digits: Before and After Training



- Untrained: all outputs ≈ 0.1, a guess
- Trained: **0.97 for "7"**; typically **97–98% test accuracy** on MNIST
- But shift the digit a few pixels and accuracy can **fall sharply**
- Cause: every pixel has its own weights, and flattening **discards neighbourhoods**
- Lecture 4: the same task with a **CNN**

Recap

- A neuron: **weighted sum + bias → activation**
- Layers of neurons form a **fully connected network**
- Four types of learning; we focus on **supervised**
- FC nets do not scale to images: **too many weights, no spatial structure**
- Next: **loss functions, backprop and optimizers**